Infrastruktur & DevOps

Table of Contents

Status . . . 1
Geplante Themen . . . 1
- CI/CD Pipeline . . . 1
- Testing-Strategie . . . 2
- Supabase Setup . . . 3
- Caching . . . 4
- Monitoring . . . 4
- Deployment . . . 5
- Environment-Variablen (geplant) . . . 6

Repository-Struktur (geplant) . . . 6
Security Best Practices (geplant) . . . 7
Disaster Recovery (geplant) . . . 7
Abhängigkeiten . . . 7
Nächste Schritte . . . 8

PLACEHOLDER — Wird bei Implementierungs-Start befüllt

Status

Diese Dokumentation wird bei Beginn der Infrastruktur-Implementierung mit spezifischen Konfigurationen, Scripts und Best-Practices gefüllt. Derzeit sind nur die geplanten Themen dokumentiert.

Geplante Themen

CI/CD Pipeline

Komponente	Beschreibung	Status
GitHub Actions	Für alle Repos (Web, Mobile, Shared)	Geplant
Typ-Generierung	Supabase → Zod → npm publish → Apps	Geplant
Zod → C++ Codegen	Für Unreal Engine Mobile App	Geplant

Komponente	Beschreibung	Status
Automated Testing	Tests auf PRs	Geplant
Staging Environment	Vor Production	Geplant
Production Environment	Live-System	Geplant

Workflows (geplant)

```
# .github/workflows/type-generation.yml
- Trigger: Supabase Schema-Änderung
- Steps:
  1. Introspect Supabase
  2. Generate Zod Schemas
  3. Generate C++ Types
  4. Publish to NPM
  5. Notify Apps
```

```
# .github/workflows/test.yml
- Trigger: Pull Request
- Steps:
  1. Unit Tests (Jest)
  2. Integration Tests (Supabase Edge Functions)
  3. E2E Tests (Playwright)
  4. Coverage Report
```

```
# .github/workflows/deploy.yml
- Trigger: Merge to main
- Steps:
  1. Build (Web)
  2. Deploy to Staging (Vercel)
  3. Smoke Tests
  4. Manual Approval
  5. Deploy to Production
```

Testing-Strategie

Test-Typ	Umfang	Tools	Beispiele
Unit Tests	Hono Services	Jest, Vitest	EffectProcessor, DCCalculator
Integration Tests	Supabase Edge Functions	Supabase Local Dev	XP-Verteilung, Sync-Operationen

Test-Typ	Umfang	Tools	Beispiele
E2E Tests	User-Flows	Playwright, Cypress	Character-Sheet Complete Flow, Combat Encounter
Snapshot Tests	UI-Komponenten	Jest	Spider-Chart, Paper-Doll, Character-Header
Performance Tests	Kritische Pfade	Lighthouse, Custom	Session-Planner Graph Load, Map Rendering

Test-Coverage-Ziele

- *Services (Business Logic):* 90%+ Coverage
- *Components (UI):* 70%+ Coverage (nur kritische)
- *Edge Functions:* 85%+ Coverage
- *Database Functions:* 80%+ Coverage

Supabase Setup

Komponente	Beschreibung	Status
RLS Policies	Kampagnen-Isolation, Spieler-Scope	Geplant
Realtime Channels	Pro Kampagne, pro Session	Geplant
Edge Functions	AI-Portraits, Discord-Webhooks, Type-Gen	Geplant
PostGIS Extension	Geographie-Layer für Maps	Geplant
Row-Level Security	Pro Rolle (GM, Spieler, Viewer)	Geplant

RLS-Struktur (geplant)

```sql
-- Kampagnen-Isolation
CREATE POLICY "Users can access own campaigns"
  ON campaigns
  USING (created_by_user_id = auth.uid());

-- Spieler dürfen nur ihre Charaktere sehen
CREATE POLICY "Players see own characters"
  ON characters
  USING (player_user_id = auth.uid() OR created_by_user_id = auth.uid());

-- GM sieht alles in der Kampagne
CREATE POLICY "GM sees all campaign data"
```

```
  ON session_mechanical_log
  USING (campaign_id IN (SELECT id FROM campaigns WHERE created_by_user_id = auth.
uid()));
```

Realtime Subscriptions (geplant)

```
// Pro-Session Channel
supabase
  .channel(`session:${sessionId}`)
  .on('postgres_changes', { event: '*', schema: 'public' }, payload => {
    // Character updates
    // NPC updates
    // Loot distribution
  })
  .subscribe();
```

Caching

Ebene	Daten	Strategy	TTL
Welt-Ebene	Rassen, Klassen, Items, Spells, Creatures	Aggressiv	1 Woche
Kampagnen-Ebene	Stories, Quests, NPCs, Organizations	Moderate	1 Stunde
Session-Ebene	Mechanical Log, Loot, Discoveries	None (Realtime)	—

Caching-Libraries

Tool	Verwendung
Redis	Distributed Cache (Supabase Sessions)
Browser LocalStorage	Client-Side UI State
Service Workers	Offline Assets + PWA

Monitoring

Aspekt	Tool	Metriken
Error Tracking	Sentry (oder PostHog)	Exception Rate, User Impact
Performance	Vercel Analytics + Custom	Page Load, API Response Time

Aspekt	Tool	Metriken
Realtime Health	Custom Dashboards	WebSocket Connections, Latency
API Metrics	Supabase Logs	Request Rate, Errors, Slow Queries

Alerting (geplant)

```
# Kritische Alerts
- Error Rate > 1%
- API Response Time > 1000ms
- Database Connections > 80%
- Edge Function Executions Failed
```

Deployment

System	Hosting	Besonderheit
Web App	Vercel oder Cloudflare Pages	Zero-downtime deploys, Edge Caching
Supabase	Managed (Official)	Auto-Backups, Updates
Hono API	Supabase Edge Functions oder Cloudflare Workers	Serverless
Mobile (Unreal)	App Store / Google Play	Distribution via App Stores

Deployment-Checklist (geplant)

```
- [ ] Tests passing (all suites)
- [ ] Type-generation successful
- [ ] Database migrations applied
- [ ] Edge Functions deployed
- [ ] Web build succeeds
- [ ] Smoke tests pass
- [ ] Performance metrics acceptable
- [ ] Security scan clear
- [ ] Manual QA sign-off
- [ ] Rollback plan documented
```

Environment-Variablen (geplant)

```
# .env.local (Development)
VITE_SUPABASE_URL=http://localhost:54321
VITE_SUPABASE_KEY=eyJ...
VITE_API_BASE_URL=http://localhost:3000

# .env.staging
VITE_SUPABASE_URL=https://staging.supabase.co
VITE_SUPABASE_KEY=eyJ...
VITE_API_BASE_URL=https://api-staging.example.com

# .env.production
VITE_SUPABASE_URL=https://prod.supabase.co
VITE_SUPABASE_KEY=eyJ...
VITE_API_BASE_URL=https://api.example.com
```

Repository-Struktur (geplant)

```
flammenreiter-monorepo/
├─── web/                        # Next.js / React Web App
│    ├─── src/
│    ├─── .github/workflows/    # CI/CD
│    └─── Dockerfile
├─── mobile/                     # Unreal Engine 5 Project
│    ├─── Source/
│    ├─── Content/
│    └─── .github/workflows/
├─── packages/
│    ├─── shared-types/         # Zod Schemas (published to npm)
│    ├─── ui-components/        # Exportable Component Library
│    └─── api-client/           # Typed Supabase Client
├─── docker/
│    ├─── Dockerfile.web
│    ├─── docker-compose.yml
│    └─── Dockerfile.functions
├─── .github/
│    ├─── workflows/
│    │    ├─── test.yml
│    │    ├─── type-generation.yml
│    │    ├─── deploy-staging.yml
│    │    └─── deploy-production.yml
│    └─── CODEOWNERS
└─── index.adoc
```

Security Best Practices (geplant)

Aspekt	Maßnahme
Secrets Management	GitHub Secrets für API Keys, DB Credentials
RLS Enforcement	Strenge Row-Level Security Policies
HTTPS Only	Alle Connections encrypted
Rate Limiting	Pro User, Pro IP
CORS Policy	Whitelist nur trusted Origins
Input Validation	Zod Schema Validation überall
SQL Injection	Parameterized Queries via ORM
CSRF Protection	SameSite Cookies
XSS Prevention	Content Security Policy Headers

Disaster Recovery (geplant)

Szenario	Maßnahme	RTO	RPO
Database Corruption	Automated Backups → Restore	30 min	1 hour
Code Deployment Bug	Automatic Rollback	5 min	0 min
Service Outage	Multi-Region Failover	10 min	5 min
Security Breach	Immediate Access Revocation	1 min	0 min

Abhängigkeiten

Abhängigkeit	Status	Auswirkung
Stabile API-Schicht (Hono)	*Erforderlich*	Basis für CI/CD
Finalisierte Repo-Struktur	*Erforderlich*	Für GitHub Actions
Supabase Account & Konfiguration	*Erforderlich*	Für Tests & Deployment
Vercel / Hosting-Provider Account	*Erforderlich*	Für Web Deployment
App Store / Google Play Accounts	*Erforderlich*	Für Mobile Deployment

---

Nächste Schritte

1. Repository-Struktur finalisieren
2. GitHub Actions Workflows implementieren
3. Supabase RLS Policies definieren und testen
4. Deployment-Prozess dokumentieren
5. Monitoring- und Alerting-System aufbauen
6. Security-Audit durchführen

Diese Dokumentation ist ein PLACEHOLDER. Sie wird bei Implementierungs-Start mit konkreten Configs, Scripts und Prozeduren befüllt. Stand: 2026-03-26